

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



FATHIR HANAN SETIAWAN

109082500198

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Algoritma

Algoritma adalah urutan langkah-langkah logis dan sistematis yang digunakan untuk menyelesaikan suatu masalah. Dalam pemrograman, algoritma diimplementasikan menggunakan bahasa pemrograman tertentu agar dapat dijalankan oleh komputer.

2. Pemrograman

Pemrograman adalah proses merancang, menulis, menguji, dan memelihara serangkaian instruksi (kode) yang diberikan kepada komputer untuk menyelesaikan tugas tertentu, menciptakan perangkat lunak, atau membangun aplikasi. Ini melibatkan penggunaan bahasa pemrograman tertentu (seperti Python, Java, atau JavaScript) untuk berkomunikasi dengan mesin agar dapat memproses data, mengotomatisasi tugas, dan memecahkan masalah secara sistematis.

3. Go (Golang)

Golang (atau Go) adalah bahasa pemrograman open-source yang dikembangkan oleh Google pada tahun 2007-2009 untuk menciptakan sistem yang cepat, sederhana, dan andal. Golang dirancang untuk menangani pengembangan aplikasi backend, sistem terdistribusi, dan cloud computing berskala besar, yang sering digunakan untuk keunggulan concurrency (pemrosesan paralel) dan performa tinggi.

4. Sorting

Sorting (pengurutan) adalah proses sistematis untuk mengatur ulang sekumpulan elemen data menjadi urutan tertentu. Proses ini bertujuan untuk mempermudah pencarian (searching), analisis, dan penyajian data. Data umumnya diurutkan secara menaik (ascending) atau menurun (descending).

5. Selection Sort

Selection sort adalah algoritma pengurutan sederhana yang bekerja dengan cara berulang kali mencari elemen nilai terkecil (atau terbesar) dari daftar data yang belum terurut, lalu menukarnya ke posisi yang tepat. Algoritma ini sangat ideal untuk dataset berukuran kecil karena kesederhanaannya.

6. Insertion Sort

Insertion sort adalah algoritma pengurutan (sorting) dasar yang bekerja dengan cara mengambil elemen satu per satu dari daftar yang belum terurut, lalu menempatkan (menyisipkan) elemen tersebut ke posisi yang tepat pada bagian daftar yang sudah diurutkan.

B. Guided

1. Selection Sort Array

```
package main

import "fmt"

func selectionSortArray(arr *[8]int) {
    var idx_min, i, j int
    for i = 0; i < len(arr); i++ {
        idx_min = i
        for j = i + 1; j < len(arr); j++ {
            if arr[j] < arr[idx_min] {
                idx_min = j
            }
        }
        if idx_min != i {
            arr[i], arr[idx_min] = arr[idx_min], arr[i]
        }
    }
}

func main() {
    var arrAngka [8]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan angka index ke-%d: ", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("=== SEBELUM SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
    fmt.Println()

    selectionSortArray(&arrAngka)
    fmt.Println("=== SETELAH SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
}
```

Output :

```

PS C:\DATA\Documents\KULIAH\Semester 2\Alpro (Praktikum)\109082500198_
\109082500198_Fathir-Hanan-Setiawan\Week 12 - Modul 14\Guided\Guided-1
Masukkan angka index ke-0: 5
Masukkan angka index ke-1: 3
Masukkan angka index ke-2: 8
Masukkan angka index ke-3: 1
Masukkan angka index ke-4: 9
Masukkan angka index ke-5: 2
Masukkan angka index ke-6: 7
Masukkan angka index ke-7: 4

=== SEBELUM SORTING ===
5 3 8 1 9 2 7 4
=== SETELAH SORTING ===
1 2 3 4 5 7 8 9
PS C:\DATA\Documents\KULIAH\Semester 2\Alpro (Praktikum)\109082500198_

```

2. Selection Sort Struct

```

package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK        float64
}

func selectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ {
        idx_min = i
        for j = i + 1; j < len(sMHS); j++ {
            if sMHS[j].IPK < sMHS[idx_min].IPK {
                idx_min = j
            }
        }
        if idx_min != i {
            sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan data mahasiswa ke-%d: ---
\n", i)
        fmt.Print("Nama : ")
    }
}

```

```

        fmt.Scan(&structMHS[i].nama)
        fmt.Print("NIM : ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK : ")
        fmt.Scan(&structMHS[i].IPK)
    }
    fmt.Println()

    fmt.Println("=== SEBELUM SORTING ===")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data mahasiswa ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()

    selectionSortArray(&structMHS)
    fmt.Println("=== SETELAH SORTING ===")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data mahasiswa ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()
}

```

Output :

```
PS C:\DATA\Documents\KULIAH\Semester 2\Alpro (Praktikum)\109082500198_
\109082500198_Fathir-Hanan-Setiawan\Week 12 - Modul 14\Guided\Guided-2
--- Masukkan data mahasiswa ke-0: ---
Nama : Andi
NIM : 10001
IPK : 3.50
--- Masukkan data mahasiswa ke-1: ---
Nama : Budi
NIM : 10002
IPK : 2.80
--- Masukkan data mahasiswa ke-2: ---
Nama : Citra
NIM : 10003
IPK : 3.75
--- Masukkan data mahasiswa ke-3: ---
Nama : Dewi
NIM : 10004
IPK : 3.10
--- Masukkan data mahasiswa ke-4: ---
Nama : Eko
NIM : 10005
IPK : 2.50
```

```
=== SEBELUM SORTING ===
--- Data mahasiswa ke-0 ---
Nama : Andi
NIM : 10001
IPK : 3.50
--- Data mahasiswa ke-1 ---
Nama : Budi
NIM : 10002
IPK : 2.80
--- Data mahasiswa ke-2 ---
Nama : Citra
NIM : 10003
IPK : 3.75
--- Data mahasiswa ke-3 ---
Nama : Dewi
NIM : 10004
IPK : 3.10
--- Data mahasiswa ke-4 ---
Nama : Eko
NIM : 10005
IPK : 2.50
=====
```

```
=== SETELAH SORTING ===
--- Data mahasiswa ke-0 ---
Nama : Eko
NIM : 10005
IPK : 2.50
--- Data mahasiswa ke-1 ---
Nama : Budi
NIM : 10002
IPK : 2.80
--- Data mahasiswa ke-2 ---
Nama : Dewi
NIM : 10004
IPK : 3.10
--- Data mahasiswa ke-3 ---
Nama : Andi
NIM : 10001
IPK : 3.50
--- Data mahasiswa ke-4 ---
Nama : Citra
NIM : 10003
IPK : 3.75
=====
```

3. Insertion Descending

```
package main

import "fmt"
```

```

type arrInt [100]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int
    for i = 1; i <= n-1; i++ {
        temp = T[i]
        j = i
        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }
        T[j] = temp
    }
}

func main() {
    var data arrInt
    var n, i int

    fmt.Print("Masukkan jumlah data : ")
    fmt.Scan(&n)

    fmt.Println("Masukkan data : ")

    for i = 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    fmt.Println("\nData sebelum sorting : ")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    insertionSort(&data, n)
    fmt.Println("\n\nData setelah sorting (Descending) : ")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    fmt.Println()
}

```

Output :

```

PS C:\DATA\Documents\KULIAH\Semester 2\Alpro (Praktikum)\109082500198_
\109082500198_Fathir-Hanan-Setiawan\Week 12 - Modul 14\Guided\Guided-3
Masukkan jumlah data : 6
Masukkan data :
4 7 2 9 1 5

Data sebelum sorting :
4 7 2 9 1 5

Data setelah sorting (Descending) :
9 7 5 4 2 1
PS C:\DATA\Documents\KULIAH\Semester 2\Alpro (Praktikum)\109082500198_

```

4. Insertion Struct

```

package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK        float64
}

type arrMHS [100]mahasiswa

func insertionSortStruct(T *arrMHS, n int) {
    var temp mahasiswa
    var i, j int
    for i = 1; i <= n-1; i++ {
        temp = T[i]
        j = i - 1
        for j >= 0 && T[j].nama < temp.nama {
            T[j+1] = T[j]
            j = j - 1
        }
        T[j+1] = temp
    }
}

func main() {
    var data arrMHS
    var n, i int

    fmt.Print("Masukkan jumlah mahasiswa : ")
    fmt.Scan(&n)

    for i = 0; i < n; i++ {
        fmt.Println("\nData Mahasiswa ke-", i+1)
    }
}

```

```
        fmt.Print("Nama : ")
        fmt.Scan(&data[i].nama)

        fmt.Print("NIM : ")
        fmt.Scan(&data[i].nim)

        fmt.Print("IPK : ")
        fmt.Scan(&data[i].IPK)
    }

    fmt.Println("\nData sebelum sorting : ")
    for i = 0; i < n; i++ {
        fmt.Println(data[i].nama, data[i].nim, data[i].IPK)
    }

    insertionSortStruct(&data, n)

    fmt.Println("\nData setelah sorting (Descending
berdasarkan nama) : ")
    for i = 0; i < n; i++ {
        fmt.Println(data[i].nama, data[i].nim, data[i].IPK)
    }
}
```

Output :

```
PS C:\DATA\Documents\KULIAH\Semester 2\Alpro (Praktikum)\109082500198_
\109082500198_Fathir-Hanan-Setiawan\Week 12 - Modul 14\Guided\Guided-4
Masukkan jumlah mahasiswa : 4

Data Mahasiswa ke- 1
Nama : Andi
NIM : 10001
IPK : 3.50

Data Mahasiswa ke- 2
Nama : Citra
NIM : 10003
IPK : 3.75

Data Mahasiswa ke- 3
Nama : Budi
NIM : 10002
IPK : 2.80

Data Mahasiswa ke- 4
Nama : Dewi
NIM : 10004
IPK : 3.10

Data sebelum sorting :
Andi 10001 3.5
Citra 10003 3.75
Budi 10002 2.8
Dewi 10004 3.1

Data setelah sorting (Descending berdasarkan nama) :
Dewi 10004 3.1
Citra 10003 3.75
Budi 10002 2.8
Andi 10001 3.5
PS C:\DATA\Documents\KULIAH\Semester 2\Alpro (Praktikum)\109082500198_
```

C. Unguided

1. Soal Unguided 1

Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu.

Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program **rumahkerabat** yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 2 7 9 13 15 27 39 75 133 189 1 4 9

Keterangan: Terdapat 3 daerah dalam contoh input, dan di masing-masing daerah mempunyai 5, 6, dan 3 kerabat.

Jawaban :

```
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        rumah := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }

        for j := 0; j < m-1; j++ {
            minIdx := j
            for k := j + 1; k < m; k++ {
                if rumah[k] < rumah[minIdx] {
                    minIdx = k
                }
            }
        }
    }
}
```

```

    }
    temp := rumah[j]
    rumah[j] = rumah[minIdx]
    rumah[minIdx] = temp
}

for j := 0; j < m; j++ {
    if j > 0 {
        fmt.Print(" ")
    }
    fmt.Print(rumah[j])
}
fmt.Println()
}
}

```

Output :

```

PS C:\DATA\Documents\KULIAH\Semester 2\Alpro (Praktikum)\109082500198_
\Week 12 - Modul 14\Unguided\unguided-no1\" ; if ($?) { go run no1.go
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS C:\DATA\Documents\KULIAH\Semester 2\Alpro (Praktikum)\109082500198_

```

Penjelasan :

Program ini dibuat untuk mengurutkan nomor rumah kerabat di setiap daerah menggunakan algoritma selection sort. Pertama, program membaca nilai n sebagai banyaknya daerah, lalu untuk setiap daerah dibaca nilai m sebagai banyaknya rumah diikuti dengan nomor-nomor rumahnya ke dalam array. Proses pengurutan dilakukan dengan mencari nilai terkecil dari elemen yang belum terurut, lalu menukarnya ke posisi paling depan, begitu terus sampai semua elemen terurut membesar. Hasil pengurutan tiap daerah langsung dicetak per baris.

2. Soal Unguided 2

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program **kerabat dekat** yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Format **Masukan** masih persis sama seperti sebelumnya.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 13 12 8 2 15 27 39 75 133 189 8 4 2

Keterangan: Terdapat 3 daerah dalam contoh masukan. Baris kedua berisi campuran bilangan ganjil dan genap. Baris berikutnya hanya berisi bilangan ganjil, dan baris terakhir hanya berisi bilangan genap.

Petunjuk:

- Waktu pembacaan data, bilangan ganjil dan genap dipisahkan ke dalam dua array yang berbeda, untuk kemudian masing-masing diurutkan tersendiri.
- Atau, tetap disimpan dalam satu array, diurutkan secara keseluruhan. Tetapi pada waktu pencetakan, mulai dengan mencetak semua nilai ganjil lebih dulu, kemudian setelah selesai cetaklah semua nilai genapnya.

Jawaban :

```
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        rumah := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }
    }
}
```

```

ganjil := []int{}
genap := []int{}

for j := 0; j < m; j++ {
    if rumah[j]%2 != 0 {
        ganjil = append(ganjil, rumah[j])
    } else {
        genap = append(genap, rumah[j])
    }
}

for j := 0; j < len(ganjil)-1; j++ {
    minIdx := j
    for k := j + 1; k < len(ganjil); k++ {
        if ganjil[k] < ganjil[minIdx] {
            minIdx = k
        }
    }
    temp := ganjil[j]
    ganjil[j] = ganjil[minIdx]
    ganjil[minIdx] = temp
}

for j := 0; j < len(genap)-1; j++ {
    maxIdx := j
    for k := j + 1; k < len(genap); k++ {
        if genap[k] > genap[maxIdx] {
            maxIdx = k
        }
    }
    temp := genap[j]
    genap[j] = genap[maxIdx]
    genap[maxIdx] = temp
}

hasil := append(ganjil, genap...)

for j := 0; j < len(hasil); j++ {
    if j > 0 {
        fmt.Print(" ")
    }
    fmt.Print(hasil[j])
}
fmt.Println()
}

```

Output :

```

PS C:\DATA\Documents\KULIAH\Semester 2\Alpro (Praktikum)\109082500198_
tikum)\109082500198_Fathir-Hanan-Setiawan\Week 12 - Modul 14\Unguided\
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS C:\DATA\Documents\KULIAH\Semester 2\Alpro (Praktikum)\109082500198_

```

Penjelasan :

Program ini memisahkan nomor rumah ganjil dan genap ke dalam dua array yang berbeda untuk setiap daerah. Array ganjil diurutkan membesar dan array genap diurutkan mengecil menggunakan selection sort, lalu keduanya digabung dan dicetak. Pemisahan dilakukan dengan mengecek sisa bagi dua dari tiap elemen saat pembacaan data. Setelah kedua array terurut sesuai ketentuannya, array ganjil dicetak lebih dulu baru diikuti array genap.

3. Soal Unguided 3

Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda *insertion sort*), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Contoh masukan dan keluaran

No	Masukan	Keluaran
1	31 13 25 43 1 7 19 37 5	1 7 13 19 25 31 37 43 Data berjarak 6
2	4 40 14 8 26 1 38 2 32 -31	1 2 4 8 14 26 32 38 40 Data berjarak tidak tetap

Jawaban :

```

package main

import "fmt"

func main() {
    data := []int{}

```

```

for {
    var x int
    fmt.Scan(&x)
    if x < 0 {
        break
    }
    data = append(data, x)
}

for i := 1; i < len(data); i++ {
    key := data[i]
    j := i - 1
    for j >= 0 && data[j] > key {
        data[j+1] = data[j]
        j--
    }
    data[j+1] = key
}

for i := 0; i < len(data); i++ {
    if i > 0 {
        fmt.Print(" ")
    }
    fmt.Print(data[i])
}
fmt.Println()

jarak := data[1] - data[0]
tetap := true
for i := 2; i < len(data); i++ {
    if data[i]-data[i-1] != jarak {
        tetap = false
        break
    }
}

if tetap {
    fmt.Println("Data berjarak", jarak)
} else {
    fmt.Println("Data berjarak tidak tetap")
}
}

```

Output :

```

PS C:\DATA\Documents\KULIAH\Semester 2\Alpro (Praktikum)\109082500198_
tikum)\109082500198_Fathir-Hanan-Setiawan\Week 12 - Modul 14\Unguided\
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS C:\DATA\Documents\KULIAH\Semester 2\Alpro (Praktikum)\109082500198_
tikum)\109082500198_Fathir-Hanan-Setiawan\Week 12 - Modul 14\Unguided\
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS C:\DATA\Documents\KULIAH\Semester 2\Alpro (Praktikum)\109082500198_

```

Penjelasan :

Program membaca bilangan bulat non-negatif sampai menemukan bilangan negatif sebagai penanda berhenti, lalu menyimpannya ke dalam array. Array kemudian diurutkan menggunakan insertion sort dengan cara mengambil elemen ke-i dan menggeser elemen-elemen sebelumnya yang lebih besar ke kanan sampai ditemukan posisi yang tepat. Setelah terurut, program mengecek apakah selisih antar elemen yang berdekatan selalu sama dengan mengambil selisih dua elemen pertama sebagai patokan jarak, lalu membandingkannya dengan selisih pasangan elemen berikutnya. Jika semua selisihnya sama maka dicetak "Data berjarak x", jika tidak maka dicetak "Data berjarak tidak tetap".

4. Soal Unguided 4

Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```

const nMax : integer = 7919
type Buku = <
    id, judul, penulis, penerbit : string
    eksemplar, tahun, rating : integer >

type DaftarBuku = array [ 1..nMax ] of Buku
Pustaka : DaftarBuku
nPustaka: integer

```

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

Lengkapi subprogram-subprogram dibawah ini, sesuai dengan I.S. dan F.S yang diberikan.

```
procedure DaftarkanBuku(in/out pustaka : DaftarBuku, n : integer)
{I.S. sejumlah n data buku telah siap para piranti masukan
 F.S. n berisi sebuah nilai, dan pustaka berisi sejumlah n data buku}

procedure CetakTerfavorit(in pustaka : DaftarBuku, in n : integer)
{I.S. array pustaka berisi n buah data buku dan belum terurut
 F.S. Tampilan data buku (judul, penulis, penerbit, tahun)
 terfavorit, yaitu memiliki rating tertinggi}

procedure UrutBuku( in/out pustaka : DaftarBuku, n : integer )
{I.S. Array pustaka berisi n data buku
 F.S. Array pustaka terurut menurun/mengecil terhadap rating.
 Catatan: Gunakan metoda Insertion sort}

procedure Cetak5Terbaru( in pustaka : DaftarBuku, n integer )
{I.S. pustaka berisi n data buku yang sudah terurut menurut rating
 F.S. Laporan 5 judul buku dengan rating tertinggi
 Catatan: Isi pustaka mungkin saja kurang dari 5}

procedure CariBuku(in pustaka : DaftarBuku, n : integer, r : integer )
{I.S. pustaka berisi n data buku yang sudah terurut menurut rating
 F.S. Laporan salah satu buku (judul, penulis, penerbit, tahun,
 eksemplar, rating) dengan rating yang diberikan. Jika tidak ada buku
 dengan rating yang ditanyakan, cukup tuliskan "Tidak ada buku dengan
 rating seperti itu". Catatan: Gunakan pencarian biner/belah dua.}
```

Jawaban :

```
package main

import "fmt"

type Buku struct {
    id          string
    judul       string
    penulis     string
    penerbit    string
    eksemplar   int
    tahun       int
    rating      int
}

var pustaka []Buku
var nPustaka int
```

```

func daftarkanBuku() {
    fmt.Scan(&nPustaka)
    pustaka = make([]Buku, nPustaka)
    for i := 0; i < nPustaka; i++ {
        fmt.Scan(&pustaka[i].id)
        fmt.Scan(&pustaka[i].judul)
        fmt.Scan(&pustaka[i].penulis)
        fmt.Scan(&pustaka[i].penerbit)
        fmt.Scan(&pustaka[i].eksemplar)
        fmt.Scan(&pustaka[i].tahun)
        fmt.Scan(&pustaka[i].rating)
    }
}

func cetakTerfavorit() {
    maxIdx := 0
    for i := 1; i < nPustaka; i++ {
        if pustaka[i].rating > pustaka[maxIdx].rating {
            maxIdx = i
        }
    }
    fmt.Println(pustaka[maxIdx].judul,
pustaka[maxIdx].penulis, pustaka[maxIdx].penerbit,
pustaka[maxIdx].tahun)
}

func urutBuku() {
    for i := 1; i < nPustaka; i++ {
        key := pustaka[i]
        j := i - 1
        for j >= 0 && pustaka[j].rating < key.rating {
            pustaka[j+1] = pustaka[j]
            j--
        }
        pustaka[j+1] = key
    }
}

func cetak5Terbaru() {
    limit := 5
    if nPustaka < 5 {
        limit = nPustaka
    }
    for i := 0; i < limit; i++ {
        fmt.Println(pustaka[i].judul)
    }
}

func cariBuku(r int) {
    lo := 0
    hi := nPustaka - 1

```

```

        ketemu := -1
        for lo <= hi {
            mid := (lo + hi) / 2
            if pustaka[mid].rating == r {
                ketemu = mid
                break
            } else if pustaka[mid].rating > r {
                lo = mid + 1
            } else {
                hi = mid - 1
            }
        }
        if ketemu == -1 {
            fmt.Println("Tidak ada buku dengan rating seperti
itu")
        } else {
            fmt.Println(pustaka[ketemu].judul,
pustaka[ketemu].penulis, pustaka[ketemu].penerbit,
pustaka[ketemu].tahun, pustaka[ketemu].eksemplar,
pustaka[ketemu].rating)
        }
    }

func main() {
    daftarkanBuku()
    cetakTerfavorit()
   urutBuku()
    cetak5Terbaru()
    var r int
    fmt.Scan(&r)
    cariBuku(r)
}

```

Output :

```

PS C:\DATA\Documents\KULIAH\Semester 2\Alpro (Praktikum)\109082500198_
tikum)\109082500198_Fathir-Hanan-Setiawan\Week 12 - Modul 14\Unguided\
7
B001 Pemrograman-Go Andi Erlangga 5 2020 90
B002 Struktur-Data Budi Gramedia 3 2019 75
B003 Algoritma Citra Informatika 8 2021 95
B004 Basis-Data Dewi Erlangga 2 2018 60
B005 Jaringan-Komputer Eko Gramedia 6 2022 85
B006 Sistem-Operasi Fani Informatika 4 2020 75
B007 Kecerdasan-Buatan Gita Erlangga 7 2023 88
Algoritma Citra Informatika 2021
Algoritma
Pemrograman-Go
Kecerdasan-Buatan
Jaringan-Komputer
Struktur-Data
75
Sistem-Operasi Fani Informatika 2020 4 75
PS C:\DATA\Documents\KULIAH\Semester 2\Alpro (Praktikum)\109082500198_

```

Penjelasan :

Program ini mengelola data buku menggunakan struct yang memiliki field id, judul, penulis, penerbit, eksemplar, tahun, dan rating, dengan semua data disimpan dalam slice global. Fungsi `DaftarkanBuku` membaca seluruh data buku dari input, lalu `CetakTerfavorit` mencari buku dengan rating tertinggi secara linear dan mencetak judul, penulis, penerbit, dan tahunnya. Fungsi `UrutBuku` mengurutkan slice secara menurun berdasarkan rating menggunakan insertion sort, kemudian `Cetak5Terbaru` mencetak maksimal 5 judul teratas dari hasil pengurutan tersebut. Terakhir, fungsi `CariBuku` menerima nilai rating dari input dan mencarinya menggunakan binary search pada array yang sudah terurut menurun, lalu mencetak data lengkap buku yang ditemukan atau pesan bahwa buku tidak ada jika tidak ketemu.

D. Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa algoritma selection sort bekerja dengan cara mencari nilai terkecil atau terbesar dari elemen yang belum terurut lalu menukarnya ke posisi yang sesuai, sedangkan insertion sort bekerja dengan cara mengambil satu elemen lalu menyisipkannya ke posisi yang tepat di antara elemen-elemen sebelumnya yang sudah terurut. Kedua algoritma ini dapat diterapkan tidak hanya pada array bertipe data primitif seperti integer, tetapi juga pada array of struct dengan menyesuaikan kondisi perbandingannya berdasarkan field tertentu seperti IPK atau nama. Selain itu, pengurutan bisa dilakukan secara ascending maupun descending tergantung dari operator perbandingan yang digunakan di dalam kondisi penukaran elemennya. Dari percobaan yang sudah dilakukan, perbedaan utama antara keduanya adalah selection sort lebih banyak melakukan penukaran sedangkan insertion sort lebih banyak melakukan pergeseran elemen.